VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY
UNIVERSITY OF SCIENCE

NHẬP MÔN HỌC MÁY

Đồ Án Cuối Kì

Bùi Công Mậu - 22127260
Thái Hữu Thọ - 22127400

GIẢNG VIÊN HƯỚNG DẪN
Thầy Bùi Duy Đăng
Thầy Huỳnh Lâm Hải Đăng
Thầy Trần Trung Kiên

Ngày 23 tháng 4 năm 2025

Mục lục

1 Giới thiệu **3**

2 Thực hiện **3**

3 Các giai đoạn thực hiện **4**
3.1 Exploratory Data Analysis (EDA) 4
3.1.1 Data collection 4
3.1.2 Data pre-processing 5
3.1.3 Feature engineering 5
3.2 Model Development 5
3.2.1 Phase 1 5
Model selection 5
Architecture design 6
Training setup 6
3.2.2 Phase 2 7
Model selection 7
Architecture design 8
Training Setup 8
3.2.3 Phase 3 9
3.2.3.1 Phase 3.1 9
3.2.3.2 Phase 3.2 11
3.3 Model Evaluation 13
3.3.1 Phase 1 13
3.3.2 Phase 2 14
3.3.3 Phase 3 15
3.3.3.1 LightGBM 15
3.3.3.2 CatBoost 15

4 Ưu điểm và nhược điểm của từng mô hình qua các giai đoạn **17**

5 Đánh giá quá trình **19**

6 Nguồn tham khảo **19**

1 Giới thiệu

Đây là dự án phát triển thực tế học máy (Practical Machine Learning Development) được cấu thành với 3 bước chính: Exploratory Data Analysis, Model Development và Model Evaluation.

2 Thực hiện

- Môi trường code: Visual Studio Code
- File code: Jupyter notebook
- File dữ liệu: Csv
- Phiên bản Python: 3.13.0
- Nguồn code được lưu trữ vào: Github, Google drive
- Các thư viện sử dụng:
  - pandas: thao tác dữ liệu
  - sklearn.preprocessing: nằm trong thư viện của scikit-learn dùng để tiền xử lý dữ liệu. Áp dụng MinMaxScaler để chuẩn hóa dữ liệu về khoảng [0,1]. Ngoài ra, áp dụng LabelEncoder để mã hóa nhãn dạng chuỗi thành số nguyên.
  - sklearn.model_selection: nằm trong thư viện của scikit-learn dùng để chia dữ liệu. Áp dụng train_test_split để chia dữ liệu thành tập train, test, validation theo tỉ lệ mong muốn.
  - sklearn.metrics: nằm trong thư viện của scikit-learn dùng để đánh giá hiệu suất mô hình của Machine Learning. Áp dụng root_mean_squared_log_error để tính độ lỗi khi xây dựng mô hình.
  - sklearn.ensemble: nằm trong thư viện của scikit-learn dùng để lựa mô hình để xây dựng sao cho phù hợp với dữ liệu cần làm. Áp dụng GradientBoosting Regressor để huấn luyện dữ liệu.
  - tensorflow: sử dụng flamework TensorFlow để huấn luyện mô hình.
  - catboost: thư viện được phát triển bởi Yandex, dùng để huấn luyện mô hình.
  - optuna: được dùng để tối ưu hóa siêu tham số, giúp tìm kiếm các giá trị tham số tối ưu cho học máy.

3 Các giai đoạn thực hiện

3.1 Exploratory Data Analysis (EDA)

3.1.1 Data collection

Bộ dữ liệu đã được chia sẵn theo tỷ lệ 6:4 (tập train với 120000 dữ liệu, tập test với 80000 dữ liệu).
Dữ liệu distinguishing công cụ 21 thuộc tính. Tuy nhiên đối với tập test thì không có thuộc tính "target" (dùng mô hình để dự đoán thuộc tính này cho tập test). Các thuộc tính được miêu tả như sau:

Thuộc tính	**Miêu tả**
id	Thứ tự của bộ dữ liệu
Age	Tuổi
Gender	Giới tính (Male/Female)
feature 0	Số liên quan đến huấn luyện dữ liệu
feature 1	Tình trạng hôn nhân (Married/Divorced/ Single)
feature 2	Số liên quan đến huấn luyện dữ liệu
feature 3	Trình độ học vấn (Bachelor's/Master's/High school/PhD)
feature 4	Tình trạng làm việc (Self Employed / Employed / Unemployed)
feature 5	Số liên quan đến huấn luyện dữ liệu
feature 6	Địa điểm làm việc (Urban/Rural/Suburban)
feature 7	(Basic/Premium/Comprehensive)
feature 8	Số liên quan đến huấn luyện dữ liệu
feature 9	Số liên quan đến huấn luyện dữ liệu
feature 10	Số liên quan đến huấn luyện dữ liệu
feature 11	Số liên quan đến huấn luyện dữ liệu
feature 12	Thời gian (Ngày, tháng, năm, giờ, phút, giây)
feature 13	Tình trạng tài chính (Poor/Average/Good)
feature 14	(Yes/No)
feature 15	(Daily/Weekly/Monthly/Rarely)
feature 16	Nhà ở (House/Condo/Apartment)

target	Giá trị liên tục (giá trị cần phải dự đoán cho tập test)

3.1.2 Data pre-processing

Qua thống kê và xử lý dữ liệu, nhận thấy rằng dữ liệu có 1 số vấn đề như sau:

- Xuất hiện missing values ở hầu hết tất cả các thuộc tính. Xử lý tùy thuộc vào kiểu dữ liệu. Đối với dữ liệu dạng số (int64, float64) sẽ điền giá trị mean. Điền giá trị bất kỳ vào các cột này có thể gây sai lệch dữ liệu và dẫn tới dự đoán không còn chính xác.
- Các cột dữ liệu dạng số (int64, float64) chưa được chuẩn hóa về khoảng giá trị nhất định. Áp dụng MinMaxScaler để chuẩn hóa tất cả các dữ liệu về khoảng giá trị $[0, 1]$.
- Các cột dữ liệu dạng chuỗi (object) chưa được mã hóa để có thể sử dụng chuyển về dạng số để áp dụng các mô hình dự đoán.

3.1.3 Feature engineering

- Để dùng mô hình dự đoán giá trị target cho test, phải dùng các dữ liệu phù hợp để làm.
- Nhận thấy rằng cột "id" không đóng vai trò quan trọng cũng như không phù hợp cho việc huấn luyện mô hình dự đoán, và cột "target" là cột cần dự đoán cho tập test nên phải bỏ khi dùng mô hình. Đặc biệt, cột "feature_12" liên quan tới thời gian (ngày, tháng, năm, giờ, phút, giây) không thể mã hóa cột này theo cách thông thường và khi bỏ đi không gây ảnh hưởng tới việc huấn luyện mô hình. Tóm lại, bỏ các cột "id", "target" và "feature_12".

3.2 Model Development

3.2.1 Phase 1

Sử dụng mô hình Gradient Boosting Regressor (Supervised learning) trong thư viện sklearn.ensemble để huấn luyện dự đoán target cho tập test.

Model selection

- Đây là thuật toán tăng cường kết hợp nhiều learner yếu để tạo ra một mô hình dự đoán mạnh.

- Nó hoạt động bằng cách train tuần tự các mô hình trong đó mỗi mô hình mới sẽ cố gắng sửa các lỗi do các mô hình trước gây ra.
- Lý do lựa chọn mô hình này:
  - Phù hợp với dataset đề bài cho dạng bảng (tabular).
  - Hiệu quả trong mô hình hóa quan hệ phi tuyến phức tạp giữa đầu ra đầu vào kết hợp nhiều learner yếu.
  - Cải thiện dần qua từng bước train: hoạt động theo cơ chế tuần tự, cây mới được tạo ra được train để sửa lỗi các tổ hợp cây trước đó. Tăng hiệu suất và giảm độ lỗi tổng thể.
  - Có thể dễ dàng điều chỉnh các siêu tham số để train mô hình.

Architecture design

- Các tham số quan trọng trong Gradient Boosting Regressor:
  - n_estimators: Số lượng cây quyết định trong mô hình.
  - learning_rate: Tỷ lệ học, ảnh hưởng đến tốc độ của mô hình.
  - random_state: Hạt giống ngẫu nhiên để tái lập kết quả.

Training setup

- Dữ liệu:
  - Train: train_normalized.csv
    - X: Các cột đặc trưng (loại bỏ id, target)
    - y: Cột target
  - Test: test_normalized.csv
    - X_test: Loại bỏ id
  - Train/Validation split: 80% huấn luyện, 20% validation
    - random_state = 42 (đảm bảo tái lập kết quả)
- Huấn luyện mô hình:
  - Thông số chính:
    - n_estimators = 3000 (số cây)
    - learning_rate = 0.1

    * random_state = 42
  - Đo hiệu suất mô hình:
    * Thời gian huấn luyện: đo bằng time.time()
    * Bộ nhớ RAM sử dụng: theo dõi với psutil.Process().memory_info().rss
  - Đánh giá:
  - Trên validation:
    * Dự đoán: y_pred_val = model.predict(X_val)
    * Đánh giá: RMSLE (Root Mean Squared Log Error)
  - Dự đoán kết quả:
    * Dự đoán: y_test_pred = model.predict(X_test)
    * Xuất file: Phase 1.csv với cột id và target

3.2.2 Phase 2

Sử dụng framework TensorFlow để thực hiện đồ án deep learning. Neural network được thiết kế với 2 lớp ẩn (64 và 32 nơ-ron, ReLU).

Model selection

- Thuật toán Deep Neural Network - DNN được huấn luyện thông qua forward propagation và backpropagation, với kiến trúc gồm các lớp như sau:
  - Lớp ẩn 1: 64 nơ-ron, với hàm kích hoạt ReLU: $\text{ReLU}(x) = \max(0, x)$
  - Lớp ẩn 2: 32 nơ-ron, ReLU.
  - Lớp đầu ra: 1 nơ-ron, dự đoán target
- Forward propagation: Dữ liệu đầu vào là 18 thuộc tính, các thuộc tính đi qua các lớp nơ-ron:
- Loss function: được tính theo MSE, do sai số logarit giữa target thực tế và dự đoán, với điều kiện rằng đảm bảo giá trị dự đoán không nhỏ hơn $0\hat{\eta} \geq 0$
- Back propagation: Tính gradient của MSE, dùng Adam (Adaptive Moment Estimation) để cập nhật trọng số.
- Dự đoán trực tiếp target.
- Lý do lựa chọn TensorFlow:
  - Xây dựng mạng nơ-rôn linh hoạt: API Keras tích hợp cho phép thiết kế DNN (2 lớp ẩn, ReLU) dễ dàng, giảm thời gian cải tiến.

  - Xử lý dữ liệu lớn: tf.data.Dataset (cache, shuffle, batch, prefetch) tối ưu cho 120.000 mẫu, giảm thời gian huấn luyện.
  - Cung cấp optimizer Adam, hàm MSE và back propagation hiệu quả.

Architecture design
Các tham số quan trọng TensorFlow:

- Epochs: Số lần toàn bộ tập dữ liệu được huấn luyện được đưa qua. Số lượng epoch cao giúp mô hình học kỹ hơn, nhưng nếu số lượng epoch quá cao có thể dẫn đến overfitting.
- batch_size: Kích thước của một batch số mẫu sẽ được đưa vào mô hình trước khi tính toán gradient và cập nhật trọng số. Batch nhỏ giúp cập nhật nhanh nhưng dao động, batch lớn thì ổn định hơn nhưng chậm và tốn bộ nhớ hơn.
- Optimizer: thuật toán tối ưu hóa giúp cập nhật trọng số trong quá trình huấn luyện. Mỗi loại có đặc tính và tốc độ hội tụ khác nhau.
- loss_function: dùng MSE để tính sai số giữa giá trị dự đoán và thực tế.
- input_shape: Xác định kích thước đầu vào cho layer đầu tiên. Phải đúng với số lượng đặc trưng (features) của dữ liệu.
- Input Layer: Với đầu vào là toàn bộ các features của dữ liệu.
- Hidden Layers:
  - Dense Layer: Bao gồm 64 neuron, sử dụng kích hoạt ReLU.
- Output Layers:
  - 1 neuron trả về 1 giá trị duy nhất - dự đoán đầu ra.

Training Setup Sau các quá trình chạy huấn luyện đã cải tiến được như sau:

- tf.data FF: 16 để huấn luyện và kiểm tra được chuyển thành dạng Dataset để có thể tải dữ liệu hiệu quả hơn, hỗ trợ batching và tiền xử lý song song.
```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu',
    input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dropout(0.3),
```

```
    tf.keras.layers.Dense(1)
])
```

- Xây dựng mô hình với cấu trúc neural network gồm 3 lớp:
  - Lớp đầu tiên: Lớp Dense với 128 lớp neuron và hàm kích hoạt ReLU. Thêm lớp Dropout và BatchNormalization để giảm overfitting.
  - Lớp Dense thứ hai: 64 neuron để tiếp tục giảm overfitting.
  - Lớp cuối cùng: Lớp Dense chỉ có 1 neuron để dự đoán log của target.
- Optimizer: Sử dụng Nadam optimizer với learning rate giảm (0.0005).
- Loss_function: Mô hình sử dụng hàm mất mát MSE (Mean Squared Error), phù hợp với bài toán hồi quy.
- Metrics: Đo lường hiệu quả bằng MAE (Mean Absolute Error).
- Early Stopping: Sử dụng callback EarlyStopping để dừng quá trình huấn luyện nếu mất mát không cải thiện sau 10 epoch liên tiếp, giúp tránh overfitting.
- Huấn luyện mô hình với dữ liệu huấn luyện, sử dụng 100 epoch tối đa, và validation data để đánh giá hiệu suất sau mỗi epoch.

3.2.3 Phase 3

3.2.3.1 Phase 3.1

Sử dụng mô hình LightGBM (Supervised learning) dựa trên phương pháp Gradient Boosting.

Model selection

- Đây là thuật toán tăng cường sử dụng phương pháp Gradient Boosting xây dựng learner mạnh bằng cách thêm learner yếu thông qua giảm dần độ dốc.
- Lý do lựa chọn mô hình này:
  - Phù hợp với dataset dạng bảng (tabular).
  - Hiệu quả cao và tốc độ huấn luyện nhanh hơn các mô hình truyền thống.
  - Hỗ trợ xử lý dữ liệu mà không cần chuẩn hóa quá nhiều.

Architecture design

- Các tham số quan trọng trong LightGBM:
  - `num_boost_round`: Số vòng huấn luyện tối đa.
  - `early_stopping_rounds`: Dừng sớm nếu sau 100 vòng không cải thiện.
  - `learning_rate`: Tốc độ học nhỏ để cập nhật trọng số chậm, giúp mô hình ổn định hơn.
  - `objective = 'regression'`: Hồi quy.
  - `metric = 'rmse'`: Sai số gốc bình phương trung bình được dùng để theo dõi trong quá trình huấn luyện.

Training setup

- Dữ liệu:
  - Train: `train_normalized.csv`
    - `X`: Các cột đặc trưng (loại bỏ `id`, `target`).
    - `y`: Cột `target`.
  - Test: `test_normalized.csv`
    - `X_test`: Loại bỏ `id`.
  - Log-transform biến `target`.
  - Train/Validation split: 80 huấn luyện, 20 validation, `random_state = 42` (đảm bảo tái lập kết quả).
- Huấn luyện mô hình:
  - Thông số chính:
    - `num_boost_round = 10000`
    - `early_stopping_rounds = 100`
    - `learning_rate = 0.01`
    - `objective = 'regression'`
    - `metric = 'rmse'`
- Đo hiệu suất mô hình:
  - Thời gian huấn luyện: Đo bằng `time.time()`.
  - Bộ nhớ RAM sử dụng: Theo dõi với `psutil.Process().memory_info().rss`.
- Đánh giá:

  - Trên validation:
    * Dự đoán: `Y_pred_val_log = model.predict(X_val, num_iteration=model.best`
    * Đánh giá: RMSLE (Root Mean Squared Log Error).

- Dự đoán kết quả:
  - Dự đoán: `Y_pred_test_log = model.predict(X_test, num_iteration=model.best_i`
  - Xuất file: `Phase 3.1.csv` với cột `id` và `target`.

3.2.3.2 Phase 3.2

CatBoost là một thuật toán gradient boosting với các cải tiến đáng kể về mặt hiệu suất và khả năng tổng quát.

Model selection

- CatBoost sử dụng một số kỹ thuật để cải thiện độ chính xác và hiệu quả của gradient boosting.
- Bao gồm các kỹ thuật tạo đặc trưng (feature engineering), tối ưu hóa cây quyết định và một thuật toán mới gọi là ordered boosting.
- Mỗi cây quyết định là một weak learner, với độ sâu (depth) và số lá tối thiểu (min_data_in_leaf) được điều chỉnh qua siêu tham số.
- Số cây (iterations) xác định số lượng weak learners được kết hợp.
- Lý do lựa chọn mô hình này:
  - Dữ liệu trong bài toán này gồm 18 đặc trưng số (đã chuẩn hóa) và không có đặc trưng phân loại. CatBoost được thiết kế để xử lý tốt dữ liệu bảng, đặc biệt với các đặc trưng số bị lệch (dù không áp dụng biến đổi logarit).
  - Sau khi loại bỏ ngoại lệ (1% trên/dưới) và áp dụng augmentation (thêm 20% dữ liệu nhiễu), tập huấn luyện có khoảng 142.560 mẫu. CatBoost được tối ưu hóa để xử lý dữ liệu lớn.
  - CatBoost thường được lựa chọn để thực hiện các bài toán hồi quy (Regression) dự đoán giá trị liên tục.

Architecture design

Các tham số quan trọng khi thực hiện CatBoost:

- Iterations (num_boost_round): Tham số xác định tổng số vòng lặp boosting, tương ứng với số lượng cây quyết định mà mô hình sẽ xây dựng trong quá trình huấn luyện.
- Depth: Quy định độ sâu tối đa cho mỗi cây quyết định được tạo ra. Nếu đặt giá trị này quá cao mà không có sự kiểm soát phù hợp, mô hình có thể dễ dàng rơi vào tình trạng quá khớp (overfitting).
- learning_rate: Đây là tốc độ học, một yếu tố kiểm soát mức độ điều chỉnh của mô hình sau mỗi vòng lặp, từ đó ảnh hưởng đến tốc độ hội tụ (học nhanh hay chậm).
- loss_function: Cho phép chỉ định hàm mất mát (loss function) mà mô hình sẽ sử dụng để đánh giá và tối ưu hóa trong quá trình huấn luyện.
- cat_features: Tham số này dùng để cung cấp một danh sách các đặc trưng (features) có bản chất là dữ liệu phân loại (categorical).
- Đầu vào sẽ là các Features và Target cần được dự đoán.
- Thực hiện vòng lặp dựa trên tham số iterations:
  - Tính residual error: $y_{residual} = y_{true} - y_{pred}$.
  - Train cây quyết định mới để dự đoán residual.
  - Cập nhật dự đoán: $y_{pred_new} = y_{pred_old}$learning_rate $\times$ tree_output
  - Dừng nếu early_stopping_rounds kích hoạt.

Training Setup

- Thông số của siêu tham số:
  - loss_function: RMSE (được CatBoost hỗ trợ trực tiếp)
  - Learning_rate: 0,03
  - Depth: 6
  - iterations: 10000
- Bởi vì các feature đã được chuẩn hóa và encoding nên không cần chọn cat_feature nữa.

- `X_train, X_val, Y_train, Y_val = train_test_split(X, Y, test_size=0.2, random_state=42)`
- Chia bộ dữ liệu thành 2 phần: tập huấn luyện và tập kiểm tra. `dtrain = cb.Pool(X_train, label=Y_train)` `dval = cb.Pool(X_val, label=Y_val)`
- Trong CatBoost, Pool được sử dụng để chứa các feature và label. Khi sử dụng cb.Pool, CatBoost có thể tối ưu hóa các tính toán.
- `msle_train = root_mean_squared_log_error(Y_train, np.clip(Y_pred_train, 0.01, None))`. Đánh giá RMSLE dựa trên validation.
- Tăng iterations, depth, giảm nhẹ learning_rate và thêm regularization, randomness để cải thiện đánh giá RMSLE.
- Sau khi cải tiến mô hình ta có các bước train mô hình như sau:
  - Xử lý ngoại lệ: các giá trị ngoại lệ của cột target được loại bỏ bằng cách loại 1% giá trị thấp nhất và 1% giá trị cao nhất.
  - Bởi vì dữ liệu phức tạp và có sự phân bố lệch lớn nên cần chuyển đổi target y theo dạng log để giảm bớt sự chênh lệch đó.
  - Biến đổi log cho các đặc trưng số bị lệch cao (skewness $>$ 0,75) để giúp mô hình học tốt hơn.
    ```
    for col in numerical_cols:
        if X[col].skew() > 0.75:
            X[col] = np.log1p(X[col].clip(lower=0))
            X_test[col] = np.log1p(X_test[col].clip(lower= 0))
    ```
  - Để giảm thiểu overfitting, một phần dữ liệu được tăng cứng bằng cách ...
  - Tối ưu hóa tham số với Optuna để tìm kiếm siêu tham số tối ưu cho mô hình. Siêu tham số như lr, depth, regularization và iteration. Sau khi đã tối ưu mô hình sẽ chọn ra siêu tham số tốt nhất để huấn luyện.

3.3 Model Evaluation

3.3.1 Phase 1

```
...  Thời gian huấn luyện: 6926.18 giây
     Mức sử dụng bộ nhớ: 334.04 MB
     RMSLE trên tập validation: 1.1483815762984104
```

- Sau khi huấn luyện ta được kết quả như sau:

  - Thời gian huấn luyện: 6926.18 giây (≈ 116 phút).
  - Mức sử dụng bộ nhớ: 334.04 MB.
  - RMSLE trên tập validation: (≈ 1.149).

- Đánh giá kết quả:
  - Tốn khá nhiều thời gian để huấn luyện.
  - Mức sử dụng bộ nhớ tương đối.
  - RMSLE cũng cao hơn thông thường.
  - Không tinh chỉnh max_depth, mặc định max_depth = 3 nên có thể chưa đủ độ sâu -> RMSLE cao.
  - Không sử dụng các kĩ thuật regularization nên RMSLE vẫn còn cao.

3.3.2 Phase 2

```
Thời gian huấn luyện: 7755.96 giây
Mức sử dụng bộ nhớ: 945.40 MB
7500/7500 ━━━━━━━━━━━━━━━━━━━━ 7s 921us/step
RMSLE trên tập test (mô hình cải tiến): 1.0750
25000/25000 ━━━━━━━━━━━━━━━━━━━━ 22s 868us/step
Đã lưu kết quả dự đoán vào file '22127260_22127400.csv'
```

- Sau khi huấn luyện ta được kết quả như sau:
  - Thời gian huấn luyện: 7755.96 giây (≈ 129 phút)
  - Mức sử dụng bộ nhớ: 945.40 MB
  - RMSLE trên tập Valid (Valid set): 1.0750

- Đánh giá kết quả:
  - Kiến trúc mô hình có cải tiến:
    * Tăng số neuron.
    * Thêm Dropout và BatchNormalization giúp giảm overfitting.
  - RMSLE = 1.0750 cao bởi vì dữ liệu lớn và có xu hướng phân bố lệch, chưa loại bỏ được hết các outliner nên target có giá trị rất lớn và rất nhỏ.
  - Các feature có giá trị gần bằng 0 mô hình bị overpredict, khó nhận biết được với các trường hợp target trung bình.
  - Các feature có độ ảnh hưởng cao khiến cho dữ liệu phân bố bị lệch.
  - Thời gian huấn luyện lâu đồng thời sử dụng nhiều RAM.

3.3.3 Phase 3

3.3.3.1 LightGBM

```
Thời gian huấn luyện: 1476.62 giây
Mức sử dụng bộ nhớ: 1064.08 MB
RMSLE trên tập validation: 1.0515759272206766
```

- Sau khi huấn luyện ta được kết quả như này:
  - Thời gian huấn luyện: 1476.6260 giây (25 phút)
  - Mức sử dụng bộ nhớ: 1064.08 MB
  - RMSLE trên tập validation: 1.0515
- Đánh giá kết quả:
  - Thời gian huấn luyện tương đối tốt
  - Mức sử dụng bộ nhớ khá cao
  - RMSLE cũng đã có cải thiện.
  - Sử dụng nhiều vòng lặp nếu không sớm có early stopping -> có thể gây overfitting.
  - Sử dụng log-transform nên dễ bị nhầm lẫn các giá trị.
  - RMSLE chỉ phản ánh độ lệch log-scale, không đánh giá độ lệch tuyệt đối.

3.3.3.2 CatBoost

```
bestTest = 0.9944029136
bestIteration = 1339

Shrink model to first 1340 iterations.
Train RMSLE: 0.9749
Validation RMSLE: 0.9944
Đã lưu file 22127260_22127400.csv với cột id và target.
Thời gian huấn luyện: 4637.44 giây
Sử dụng bộ nhớ: 631.96 MB
```

- Sau khi huấn luyện xong ta được kết quả như sau:
  - `bestTest` = 0.9944029136
  - `bestIteration` = 1339
  - Shrink model to first 1340 iterations.
  - Validation RMSLE: 0.9944
  - Thời gian huấn luyện: 4637.44
  - Sử dụng bộ nhớ: 631.96
  - Số iteration tốt nhất: 1339 (rút gọn còn 1340 iteration).
- Đánh giá kết quả:
  - RMSLE validation gần với train chứng tỏ mô hình học tốt và kernel không bị overfitting.
  - Biến đổi log cho target giúp giảm skew.
  - RMSLE vẫn còn gần 1.0: Điều này cho thấy mô hình chưa đạt độ chính xác cao, đặc biệt với các giá trị nhỏ hoặc lớn có thể sai lệch nhiều trên log-scale.
  - Có thể là do augmentation và số lượng iteration lớn (1340) dẫn đến thời gian huấn luyện vẫn còn lâu.
  - Bởi vì có nhiều feature gần như không đóng góp cho quá trình huấn luyện mô hình nên dẫn đến sai số khi học.

4 Ưu điểm và nhược điểm của từng mô hình qua các giai đoạn

Bảng 2: Ưu và nhược điểm của các mô hình

STT	Mô hình	Ưu điểm	Nhược điểm
1	Phase 1: Gradient Boosting Regressor	• Độ chính xác cao do là mô hình ensemble mạnh kết hợp nhiều cây lại. • Có thể kiểm soát overfitting thông qua điều chỉnh tham số. • Tốt với dữ liệu dạng bảng (tabular).	• Huấn luyện chậm • Không tối ưu cho CPU/RAM • Không xử lí được các dữ liệu dạng phân loại trực tiếp (phải encode mới thực hiện được).
2	Phase 2: TensorFlow	• Khả năng mở rộng: Tensor Flow có khả năng mở rộng tốt, có thể xử lý các bộ dữ liệu lớn và huấn luyện mô hình deep learning trên phần cứng 1 cách nhanh chóng. TensorFlow phù hợp với các bài toán quy mô lớn. • Linh hoạt: Tensor Flow mang lại sự linh hoạt cao trong việc thiết kế mô hình. Dễ dàng thiết kế kiến trúc mô hình phù hợp mục đích và mô hình huấn luyện. Có thể tùy chỉnh các lớp Dense, Dropout, BatchNormalization và các lớp khác nhau. • Sử dụng được nhiều công cụ hỗ trợ khác như Keras,... • Đồng thời TensorFlow hỗ trợ tính toán phân tán, giúp huấn luyện mô hình trên nhiều GPU, làm giảm đáng kể thời gian huấn luyện.	• Độ phức tạp cao bởi vì tính linh hoạt cao với các lớp tự tùy chỉnh nên khó để cấu hình và sửa lỗi. • Mô hình Deep learning nên thường yêu cầu thời gian huấn luyện lâu, lâu hơn khi là trên các bộ dữ liệu lớn. • Nguy cơ overfitting: Tuy có Dropout và BatchNormalization nhưng mô hình deep learning vẫn dễ bị overfitting. Cần phải thực hiện regulization tốt nhất để mô hình có thể huấn luyện đủ và tổng quát tốt trên các dữ liệu chưa thấy.

STT	Mô hình	Ưu điểm	Nhược điểm
3	Phase 3.1: LightGBM	• Tốc độ huấn luyện nhanh • Tiết kiệm bộ nhớ hơn rất nhiều • Hỗ trợ cả các dữ liệu dạng phân loại (không cần encode) • Có độ chính xác khá cao • Hỗ trợ early stopping để tăng hiệu quả huấn luyện	• Dễ gây overfitting nếu không điều chỉnh thông số phù hợp. • Không ổn định với dữ liệu không đủ lớn.
4	Phase 3.2: CatBoost	• CatBoost học rất tốt trong việc xử lý dữ liệu phân loại mà không hóa. Sau mỗi cây sẽ cải thiện kết quả của cây trước đó để kết quả tốt hơn. • CatBoost có tốc độ huấn luyện nhanh, thường nhanh hơn các thuật toán boosting khác như XGBoost và LightGBM thông qua khai thác quá trình boosting đặc biệt. • CatBoost sử dụng cây đối xứng. Ở mọi độ sâu của quá trình học quyết định đều sử dụng cùng 1 điều kiện chia tách. • So với các framework deep learning, CatBoost yêu cầu ít bước xử lý dữ liệu hơn nên việc triễn khai trở nên nhanh chóng và dễ dàng hơn. • Kết quả sau khi học được thông qua mô hình CatBoost được giải thích rõ ràng dễ hiểu thông qua các chỉ số quan trọng	• Với tập dữ liệu quá lớn các mô hình deep learning khác phù hợp hơn bởi vì CatBoost thường hay bỏ qua các đoạn dữ liệu dẫn đến bỏ sót thông tin làm sai lệch dự đoán. • CatBoost ít linh hoạt khi làm việc với các tập dữ liệu phức tạp như hình ảnh hay văn bản. • Cần phải tối ưu hóa siêu tham số rất nhiều để đạt được hiệu suất tốt nhất. Tuy nhiên có thể sử dụng Optuna để tử điingj háo quá trình đó.

5 Đánh giá quá trình

No.	Details	Complete
1	Phase 1: Scikit-learn	100%
2	Phase 2: Choose one or more framework between PyTorch and TensorFlow	100%
3	Phase 3: Choose at least two frameworks among XGBoost, LightGBM, and CatBoost	100%
4	Final Project Report	100%

6 Nguồn tham khảo

- ArcGIS: `https://pro.arcgis.com/en/pro-app/3.3/tool-reference/geoai/how-catboostworks.htm`
- Neptune.ai: `https://neptune.ai/blog/when-to-choose-catboost-over-xgboost-or-lightgbm`
- LightGBM: `https://www.geeksforgeeks.org/lightgbm-light-gradient-boosting-machine/`
- Gradient Boosting in ML: `https://www.geeksforgeeks.org/ml-gradient-boosting/`
- Gradient Boosting: `https://viblo.asia/p/gradient-boosting-tat-tan-tat-ve-thuat-toan-manh-me-nhat-trong-machine-learning-YWOZrN7vZQ0`
- Graph neural networks in TensorFlow: `https://blog.tensorflow.org/2024/02/graph-neural-networks-in-tensorflow.html?hl=vi`
- Hệ thống kiến thức Machine Learning: `https://nam157.github.io/ml_basis/`